

Stitch: Enhancing Selective Prediction for Improved Performance and Reliability in LLMs

Anonymous COLING 2025 submission

Abstract

In large language models (LLMs), selective prediction aims to identify which predictions are reliable to improve decision-making in critical applications. However, existing selective prediction methods often have limited applicability and fail to effectively improve model performance due to resource-intensive deployment and data bias, which is particularly dangerous in high-risk areas. To address these challenges, we propose Stitch, an innovative approach aimed at improving the performance and reliability of LLMs. Stitch enhances model capabilities through multi-task fine-tuning, combines auxiliary models for comprehensive evaluation, and utilizes RoBERTa-based reliability assessment to measure the credibility of predictions. Experimental results show that Stitch can effectively reduce output errors and significantly improve the overall reliability of the model. The Stitch is open-source and available for use and further development on GitHub at <https://github.com/Chaochao2020/Stitch>.

1 Introduction

Large language models (LLMs) have demonstrated excellent capabilities in natural language understanding, generation, and reasoning by training on large-scale datasets. These models can generate fluent and contextual texts and are widely used in fields such as chatbots, translation, and content creation. However, existing research (Yang et al., 2024; Abhimanyu Dubey et al., 2024) mainly focuses on the task performance of LLMs. In fact, due to differences in training data and model architecture, LLMs may sometimes produce wrong outputs. For example, Figure 1 shows a user asking a LLM a safety and health question: "Which vitamin assists blood coagulation?" The LLM incorrectly answers "Vitamin C" when the correct answer should be "Vitamin K". This error may cause the user to take the wrong medication, causing potential harm. Therefore, in addition to focusing

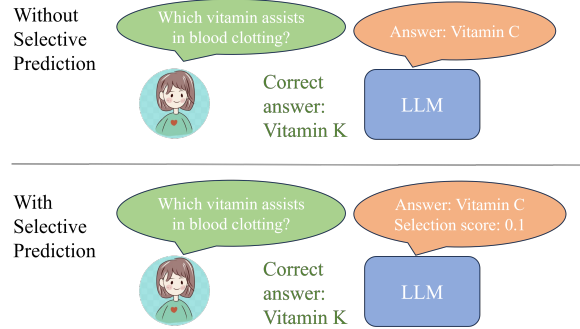


Figure 1: A LLM incorrectly answered "Vitamin C" to the safety and health question, "Which vitamin assists in blood clotting?" when the correct answer is "Vitamin K."

on the performance of LLMs on tasks, it is also necessary to deeply evaluate the reliability of their outputs to avoid errors and promptly alert users when predictions are wrong.

Recent research (Kadavath et al., 2022; Varshney et al., 2022; Wang et al.; Kuhn et al.; Chen et al., 2023) have started to focus on these issues, exploring methods such as prompt optimization, semantic entropy, self-consistency, and self-evaluation to measure the reliability of LLM outputs better. Among these methods, Parameter-Efficient Fine-tuning techniques (PEFT) (Houlsby et al., 2019) are widely adopted. PEFT techniques enhance model efficiency and performance by fine-tuning a small subset of parameters while freezing most of the pre-trained LLM's parameters, thereby significantly reducing computational and storage costs. However, existing methods face several challenges. First, they often introduce significant computational costs and latency, which limit their broader applicability. For instance, Wang et al. requires generating multiple output sequences to estimate uncertainty for a given input, and the heuristic prompts proposed by Kadavath et al., 2022 may not generalize well to other LLMs (e.g., Mistral and LLaMA3, which are

used in our work). Second, although fine-tuning can improve model performance, during inference, since LLM generates output sequences by autoregression, it may still generate wrong answers with high confidence. In addition, traditional single-task fine-tuning usually focuses on a specific task, and the fine-tuned model functions are difficult to transfer to other tasks, and it is difficult to maintain the same high performance in different tasks. Finally, previous studies ignore or do not fully address the issue of evaluating the reliability of model output while improving its reliability. In practical applications, enhancing the accuracy and reliability of model output is crucial to making more reliable decisions.

To address these challenges, we propose a novel approach called *Stitch*, designed to enhance the performance and efficiency of LLMs. Specifically, stitch improves the reliability of LLM outputs through multi-task fine-tuning, model integration, and reliability assessment. First, the Primary LLM is fine-tuned on multiple tasks to boost performance and generalization. Then, auxiliary models are integrated to mitigate the limitations of a single model. Finally, a RoBERTa-based scorer assesses the output’s reliability, with higher scores indicating greater trustworthiness. Our approach is ideal for high-risk decision-making, offering a quantitative confidence indicator to filter reliable outputs and reduce usage risks. In summary, our contributions are as follows:

- We introduce a novel approach called *Stitch*, which combines multi-task fine-tuning and model integration, designed to enhance the performance and adaptability of LLMs in diverse applications.
- We employ a RoBERTa-based scorer to quantify output reliability, avoiding the generation of multiple output sequences and significantly reducing computational overhead while improving performance and generality.
- Extensive experiments on three question-answering datasets show that *Stitch* achieves superior selective prediction performance, using Qwen2, Mistral-v0.2, and LLaMA3 models.

2 Related Work

2.1 Selective Prediction for LLMs

LLMs have achieved remarkable success in natural language generation (NLG) tasks, but they can still

produce wrong outputs. Given this problem, selective prediction is crucial in the safe and practical deployment of LLMs, as shown in Figure 1. Notably, selective prediction in NLG differs fundamentally from classification tasks like natural language inference due to the multi-step autoregression and infinite possible outputs involved. Recent studies have introduced various uncertainty measures as the basis for selective prediction (Si et al., 2023; Kadavath et al., 2022; Varshney et al., 2022; Ren et al., 2022; Kuhn et al.). These methods employ various techniques, including perplexity, semantic entropy, self-evaluation, and enhancing selective prediction through task-specific fine-tuning. However, these methods face many challenges, such as incurring high computational cost and latency, insufficient generalization of heuristic cues, autoregressive generation that may produce high-confidence wrong answers despite improved performance through fine-tuning, and often overlooking the critical need to evaluate the reliability of model output while improving output quality.

2.2 Adaptation of LLMs

Due to the large parameter size of LLMs, fine-tuning them for downstream tasks is extremely challenging for two main reasons. First, fine-tuning requires creating a new LLM copy for each specific downstream task, which is impractical for many application scenarios. Second, fine-tuning LLMs requires a large amount of device memory and can only be achieved with high-end GPUs or TPUs. To address these issues, Parameter-Efficient Fine-Tuning (PEFT) (Houlsby et al., 2019) techniques such as LoRA (Hu et al., 2022) have been proposed, which greatly alleviate the storage pressure by reducing the amount of parameters to be fine-tuned. However, these methods usually target a single task, resulting in high resource consumption and inefficient deployment.

3 Problem Setup

Consider a pre-trained LLM f designed for generative tasks, such as question answering. The LLM’s output can be represented as a sequence of tokens, where token is a unit of text, such as a word, sub-word, or character, drawn from the vocabulary $\mathcal{V}$. Define $\mathcal{V}^*$ as the space of all possible token sequences. Let $\bar{f}(v | \mathbf{x})$ denote the logits produced by f for a token $v \in \mathcal{V}$ given an input sequence $\mathbf{x} \in \mathcal{V}^*$. The probability of the token v being the

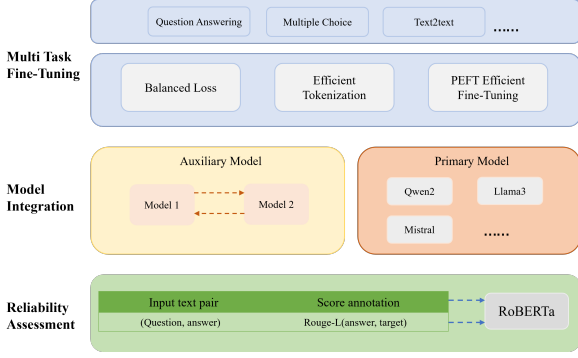


Figure 2: Overall architecture of our proposed Stitch.

next token following $\mathbf{x}$ is given by:

$$f(v | \mathbf{x}) := \frac{\exp(\bar{f}(v | \mathbf{x}))}{\sum_{v' \in \mathcal{V}} \exp(\bar{f}(v' | \mathbf{x}))}, \quad (1)$$

Similarly, the probability of generating a sequence $\hat{\mathbf{y}} \in \mathcal{V}^*$ given $\mathbf{x}$ is expressed as:

$$f(\hat{\mathbf{y}} | \mathbf{x}) := \prod_{i=1}^{|\hat{\mathbf{y}}|} f(\hat{y}_i | \mathbf{x}, \hat{y}_{[i-1]}), \quad (2)$$

where $\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{|\hat{\mathbf{y}}|})$, and $|\hat{\mathbf{y}}|$ denotes the length of the sequence $\hat{\mathbf{y}}$.

To generate the optimal output sequence for a given input $\mathbf{x}$, the model f aims to solve the following optimization problem:

$$\hat{\mathbf{y}}^* = \arg \max_{\hat{\mathbf{y}}} \log f(\hat{\mathbf{y}} | \mathbf{x}) \quad (3)$$

To assess the correctness of the generated output $\hat{\mathbf{y}}$, we use a set of reference outputs S along with an evaluation metric $M : \mathcal{V}^* \times \mathcal{V}^* \rightarrow [0, 1]$ that quantifies the similarity between the generated output $\hat{\mathbf{y}}$ and a reference output $\mathbf{y}_r \in S$. By applying a threshold γ , we can determine whether the generated output is deemed correct.

4 Stitch Approach

We argue that LLMs should be able to assess the reliability of their outputs. While prior studies show LLMs have some self-evaluation ability, this skill often lacks transferability across tasks. Moreover, a single model faces challenges in both self-evaluation and performance improvement within a specific domain.

To address this, we propose *Stitch*. As shown in Figure 2, Stitch first fine-tunes the primary model

on multiple tasks to boost performance and generalization, then incorporates auxiliary models to overcome the limitations of a single model. Finally, a RoBERTa-based scorer evaluates the reliability of the generated output. The following sections detail how Stitch improves cross-task performance and enhances LLM self-evaluation.

4.1 Multi Task Fine-Tuning

While the pre-training of LLMs is designed to enhance generalization, subsequent fine-tuning is often restricted to specific tasks. Traditional single-task fine-tuning methods are resource-intensive and inefficient for deployment. Hence, multi-task fine-tuning becomes crucial, enabling the handling of multiple tasks simultaneously and leveraging the synergy between tasks to improve overall performance. However, multi-task fine-tuning faces challenges such as data imbalance, varying task difficulty, and inconsistent convergence rates. Drawing on the work of Liu et al. (2024), we utilize LoRA (Hu et al., 2022) for multi-task fine-tuning to improve the selective prediction performance of LLMs. We have designed a specialized loss function that allows us to adaptively balance the learning progress across different tasks, ensuring that the model achieves optimal convergence across all tasks.

To prevent the model from biasing towards sub-tasks with larger data volumes, we introduced a weight assignment strategy in the loss calculation. Specifically, we assign weights based on the number of valid tokens involved in the loss calculation. The weighted loss function is shown in Equation 4. θ represents the parameters of the model that we are optimizing during training. N represents the total number of tasks, M_i denotes the number of samples for the i -th task, T_{ij} signifies the count of valid tokens (i.e., tokens involved in loss calculation) for the j -th sample of the i -th task, and t_{ijk} refers to the k -th valid token of the j -th sample for the i -th task.

$$\mathcal{L}(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N \frac{\sum_{j=1}^{M_i} \sum_{k=1}^{T_{ij}} - \log(p_{\theta}(t_{ijk}))}{\sum_{j=1}^{M_i} T_{ij}} \quad (4)$$

To address the challenge of varying task difficulty, we implemented two levels of loss functions to accommodate different granularities. One operates at the sample level, as shown in Equation 5, while the other operates at the task level, as shown

in Equation 7. This two-tiered approach allows for more precise balancing of the learning process across tasks, ensuring that the model can make balanced progress on tasks of varying difficulty levels.

$$\mathcal{L}_2(\theta) = \min_{\theta} \frac{\sum_{i=1}^N \sum_{j=1}^{M_i} -\alpha_i * (1 - P_{ij})^{\gamma} * Q_{ij}}{\sum_{i=1}^N M_i}, \quad (5)$$

$$P_{ij} = \frac{1}{T_{ij}} \sum_{k=1}^{T_{ij}} P_{ijk}, Q_{ij} = \frac{1}{T_{ij}} \sum_{k=1}^{T_{ij}} \log(P_{ijk}), \quad (6)$$

$$\mathcal{L}_3(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N -\alpha_i * (1 - P_i)^{\gamma} * Q_i, \quad (7)$$

$$P_{ij} = \frac{1}{M_i} \sum_{j=1}^{M_i} \frac{1}{T_{ij}} \sum_{k=1}^{T_{ij}} P_{ijk}, \quad (8)$$

$$Q_{ij} = \frac{1}{M_i} \sum_{j=1}^{M_i} \frac{1}{T_{ij}} \sum_{k=1}^{T_{ij}} \log(P_{ijk}) \quad (9)$$

To tackle the issue of differing convergence rates between tasks, we adopted a dynamic weight adjustment strategy. We assign an independent original loss function $\mathcal{L}^i(\theta)$ to each task i , and dynamically update the task weight according to the gradient of the validation set loss for each task at the t -th iteration. The optimization objective is to maximize the weight w^i of the slowest-converging task. This process is expressed in Equation 10.

$$\mathcal{L}_4(\theta) = \max_{g_t} \min_i \frac{1}{\alpha} c^i(\alpha, g_t) - \frac{1}{2} \|g_t\|^2, \quad (10)$$

$$g_t = \sum_i w_t^i \nabla \mathcal{L}^i(\theta_t), \quad (11)$$

$$c^i(\alpha, g_t) = \frac{\mathcal{L}^i(\theta_t) - \mathcal{L}^i(\theta_t - \alpha g_t)}{\mathcal{L}^i(\theta_t) + \epsilon} \quad (12)$$

In addition to these strategies for multi-task fine-tuning, we also focus on efficient tokenization, which plays a crucial role in the pre-training and fine-tuning of LLMs. Tokenization splits the text into small units and, together with the loss function, determines how the training data is used, significantly impacting model effectiveness and training efficiency.

As shown in Figure 3(a), the traditional supervised fine-tuning (SFT) segmentation scheme aligns the same batch of samples uniformly to the maximum input length of the model, with shortfalls made up by filler tokens. However, this approach often leads to a large number of useless padded tokens, reducing training efficiency and wasting storage space.

To address this issue, we adopt and optimize the Pack mode to maximize the use of the model's input window length. As illustrated in Figure 3(b), this mode is similar to the SFT splitter mode of Llama3 (Abhimanyu Dubey et al., 2024). In the Pack mode, multiple fine-tuning samples are sequentially packed into a fixed-length window, with neighboring samples separated by an EOS Token. When a sample does not fit completely into the current window, it is moved to the next window, and the remaining space is filled with filler Tokens. This approach significantly reduces the filler tokens ratio, improving both training efficiency and storage utilization.

By combining these advanced multi-task fine-tuning strategies with efficient tokenization techniques, we have developed a comprehensive approach that addresses the challenges of multi-task learning while maximizing computational efficiency. This integrated method allows us to leverage the full potential of LLMs across a diverse range of tasks, paving the way for more effective and resource-efficient language model deployment.

4.2 Model Integration

Single models often lack comprehensive knowledge and often have difficulties with reasoning or factual accuracy. To overcome these limitations, we developed a model integration that combines a primary model with two auxiliary models. The primary model excels in semantic understanding and reasoning, while the auxiliary model focuses on specific areas such as mathematics and code analysis. This combination ensures that the system can take full advantage of the different models, significantly improving the quality of the overall output.

Our integration approach optimizes response quality through multiple rounds of interactions. The initial response is generated by the primary model, while the secondary model provides iterative feedback through multiple rounds of dialogue, gradually refining the response. In our experiments, we found that three to five dialogue rounds can sig-

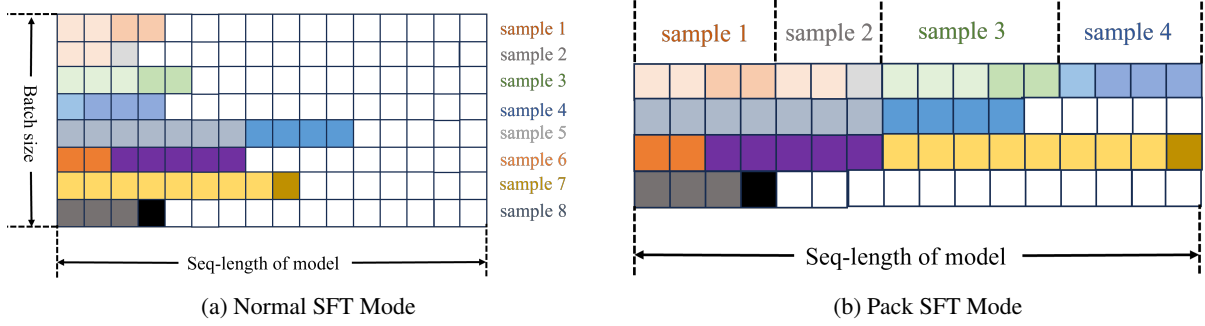


Figure 3: An illustration of the differences in sample organization between Normal SFT and Pack SFT within a batch. The light-colored squares in the figure represent the prompt portion of the sample, the dark-colored squares represent the label portion, and the blank squares represent the padding portion.

nificantly improve the accuracy and completeness of the output, while more dialogue rounds bring diminishing returns and increased computational costs. Therefore, choosing three to five dialogue rounds as the hyperparameter N_{rounds} is a reasonable solution to achieve the best balance between performance and efficiency. At the same time, using two auxiliary models can effectively balance the needs of different perspectives and computing efficiency, avoid the complexity and additional computing overhead caused by adding too many models, and provide a feedback mechanism that is both powerful and efficient.

4.3 Reliability Assessment

In order to evaluate the reliability of the model output, we introduced a pre-trained scoring model based on RoBERTa (Tan et al., 2024), which has 36 million parameters and can receive instructions and candidate responses as input, and output between 0 and 1 the score, represents the estimate of the correctness of the response relative to the instruction. The novelty of our approach is that this RoBERTa model not only significantly improves scoring accuracy by fine-tuning task-specific datasets, but also works in conjunction with a large language model (LLM) without accessing or modifying the parameters of the LLM. This makes our evaluation framework highly adaptable and compatible with various closed-source LLMs while avoiding backpropagation of LLM parameters, thereby significantly reducing memory requirements and improving deployment efficiency.

In terms of technical details, we further enhance RoBERTa’s selective prediction capabilities by constructing a high-quality training dataset consisting of carefully curated question-answer pairs. Each

pair of data is annotated based on Rouge-L (Lin and Och, 2004) to ensure that the model can effectively capture the correlation and completeness between instructions and responses.

5 Experiments

Our experimental evaluation is focused on the following questions:

(Q1) How does multi-task fine-tuning affect LLMs in selective prediction tasks?

(A1) Multi-task fine-tuning improves performance and efficiency over single-task, especially with imbalanced data and diverse task difficulties. (see Table 1)

(Q2) Can model integration improve the accuracy and quality of language model outputs?

(A2) Integrating a primary model with two auxiliary models that iteratively refine outputs significantly boosts accuracy, consistency, and overall quality, reducing issues like hallucinations and logical errors. (see Table 3, Figure 4 and Table 5.)

(Q3) How does the Stitch compare with other selective prediction methods?

(A3) The Stitch excels in reliability and safety compared to other methods, thanks to its multi-task fine-tuning, model integration, and reliability scoring, making it highly effective for critical decisions. (see Table 2)

5.1 Setup

5.1.1 Dataset

We used three question answering datasets: CommonsenseQA (Talmor et al., 2019), Ai2_arc_easy (Clark et al., 2018) and SQuAD (Rajpurkar et al., 2016).

- CommonsenseQA is a multiple-choice question answering dataset that requires different

Model	Type	Commonsense QA	Ai2_arc_easy	SQuAD	
		Acc $\uparrow$	Acc $\uparrow$	Em $\uparrow$	F1 $\uparrow$
Qwen2-0.5B Instruct	Pre-trained	0.3514	0.3889	0.1174	0.3274
	Single	0.6060	0.6595	0.6643	0.8039
	Mix	0.5758	0.6280	0.6480	0.7910
	Mft	0.6192	0.6604	0.6658	0.8095
Qwen2-7B Instruct	Pre-trained	0.7740	0.9078	0.4953	0.6997
	Single	0.8460	0.9453	0.7370	0.8730
	Mix	0.8305	0.9327	0.7253	0.8622
	Mft	0.8469	0.9490	0.7277	0.8659
Mistral-7B Instruct-v0.2	Pre-trained	0.6626	0.8106	0.1189	0.3854
	Single	0.8223	0.8986	0.7398	0.8737
	Mix	0.8125	0.8956	0.7322	0.8700
	Mft	0.8280	0.9137	0.7305	0.8681
Llama3-8B Instruct	Pre-trained	0.6740	0.8245	0.6665	0.8303
	Single	0.8182	0.9301	0.7170	0.8596
	Mix	0.8043	0.9057	0.7278	0.8665
	Mft	0.8215	0.9356	0.7293	0.8674

Table 1: Evaluation results for Acc, EM, and F1 of different LLMs. In "Type," "Pre-trained" refers to using the model without fine-tuning, "Single" to fine-tuning on a single dataset, "Mix" on a mixed dataset, and "Mft" on multi-task fine-tuning. **Bold** numbers indicate the best results.

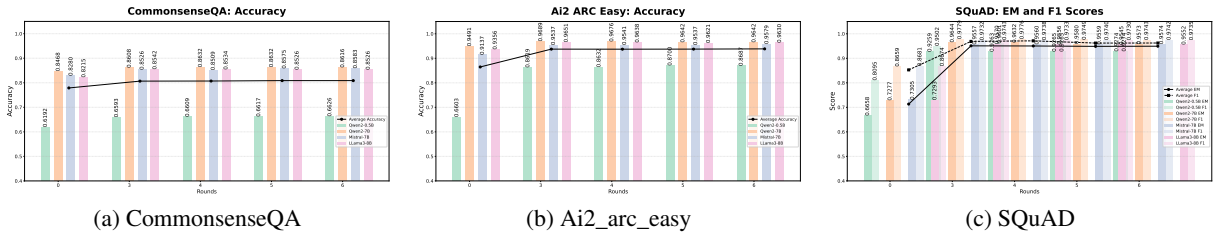


Figure 4: Impact of dialogue rounds in three datasets.

types of commonsense knowledge to predict the correct answers. It contains 12,102 questions with one correct answer and four distractor answers.

- Ai2_arc_easy is a dataset of 7,787 genuine grade-school level, multiple-choice science questions, assembled to encourage research in advanced question-answering.
- SQuAD is a reading comprehension dataset containing 100,000+ question-answer pairs on 500+ articles.

We selected three datasets in the fields of common sense, scientific problems, and reading comprehension, aiming to use multi-task fine-tuning to improve the overall performance of the llms. The LLMs can strengthen its reasoning ability on scientific problems by understanding the logical relationships in common sense, while effectively extracting information in reading comprehension

tasks, thereby improving the performance of all tasks.

5.1.2 LLMs

We used Qwen2(Yang et al., 2024), Mistral(Jiang et al., 2023) and Llama3(Abhimanyu Dubey et al., 2024) models of various sizes. For Qwen2, we considered Qwen2-7B-Instruct and Qwen2-0.5B-Instruct. For Mistral, we considered Mistral-7B-Instruct-v0.2. For Llama3, we considered Llama3-8B-Instruct.

5.1.3 Baselines

We considered the following baselines to compute the selection score: (1) Perplexity; (2) Semantic Entropy(Kuhn et al.); (3) Self-eval; (4) ASPIRE(Chen et al., 2023).

- **Perplexity**: defined as the exponentiated average negative log-likelihood of a sequence.
- **Semantic entropy**: an entropy-based uncer-

Model	Method	Commonsense QA		Ai2_arc_easy		SQuAD	
		AUACC ↑	AUROC ↑	AUACC ↑	AUROC ↑	AUACC ↑	AUROC ↑
Qwen2-0.5B Instruct	Perplexity	44.78	73.96	79.32	68.14	82.13	67.21
	Semantic Entropy	44.73	73.61	78.95	67.77	81.69	66.40
	Self-eval	28.34	50.41	69.45	45.68	74.47	48.95
	ASPIRE	27.84	50.17	71.22	41.45	69.29	45.43
	Stitch (ours)	46.72	75.27	80.79	69.16	83.60	69.13
Qwen2-7B Instruct	Perplexity	55.08	84.25	93.05	80.63	95.08	83.41
	Semantic Entropy	55.10	84.06	93.48	81.01	95.43	84.22
	Self-eval	32.48	58.53	83.55	57.18	88.56	64.91
	ASPIRE	34.98	58.77	83.39	57.43	84.81	57.40
	Stitch (ours)	55.56	86.94	94.88	87.27	96.94	90.48
Mistral-7B Instruct-v0.2	Perplexity	49.23	79.85	85.47	73.58	88.65	72.32
	Semantic Entropy	49.18	79.50	85.10	73.21	88.21	71.51
	Self-eval	31.89	54.77	75.64	49.43	80.56	53.92
	ASPIRE	31.39	54.53	75.48	49.18	77.31	46.42
	Stitch (ours)	52.83	81.63	86.99	74.91	89.70	74.80
Llama3-8B Instruct	Perplexity	50.17	80.99	87.21	74.86	89.58	73.88
	Semantic Entropy	50.12	80.64	86.84	74.49	89.14	73.07
	Self-eval	32.83	55.91	77.39	50.71	81.49	54.48
	ASPIRE	32.33	56.67	77.23	50.46	78.24	46.98
	Stitch (ours)	53.77	82.77	88.74	76.19	90.45	75.64

Table 2: Results of evaluating different methods to compute the selection score. All numbers are percentages. **Bold** numbers are superior results.

tainty measure that uses a bidirectional entailment algorithm for marginalizing over semantically equivalent samples.

- **Self-eval**: a simple baseline that obtains a selection score from the LLM by asking whether the proposed answer $\hat{y}^*$ is correct or not.
- **ASPIRE**: ASPIRE improves selective prediction by fine-tuning model parameters on downstream tasks and incorporating a self-evaluation mechanism to distinguish between correct and incorrect predictions.

5.1.4 Evaluation metrics

We used the Rouge-L (Lin and Och, 2004) metric to evaluate the correctness of predicted answers. Rouge-L measured the longest common subsequence (LCS) between the predicted and reference texts, producing a score in the range [0, 1]. To determine whether a predicted answer was correct, we compared this score against a threshold γ . If the Rouge-L score exceeded the threshold γ , the predicted answer was considered correct; otherwise, it was deemed incorrect. The Exact Match (EM) metric measured whether the predicted answer exactly matched the reference answer. The F1 score combined precision and recall into a single metric, balancing the trade-off between them, with values ranging from 0 to 1, where higher scores indicated better performance. The Area Under the

Model ($N_{rounds} = 3$)	Comm- onsense QA	Ai2_arc_easy	SQuAD	
	Acc ↑	Acc ↑	Em ↑	F1 ↑
Qwen2-0.5B Instruct	0.6601	0.8620	0.9259	0.9502
Qwen2-7B Instruct	0.8608	0.9689	0.9644	0.9779
Mistral-7B Instruct-v0.2	0.8526	0.9537	0.9557	0.9732
Llama3-8B Instruct	0.8542	0.9651	0.9570	0.9743

Table 3: The effect of model integration on the accuracy and quality of LLMs output.

Accuracy Curve (AUACC) evaluated overall accuracy by plotting accuracy against sample size and calculating the area under the curve. Similarly, the Area Under the Receiver Operating Characteristic Curve (AUROC) measured a model’s ability to distinguish between classes by calculating the area under the ROC curve, with values also ranging from 0 to 1, where higher values indicated better class discrimination.

5.2 Results

We presented a comprehensive evaluation of our methods, focusing on the impact of multi-task fine-tuning, comparisons with baseline methods, ablation studies, and parameter effects.

- We evaluated model performance based on

fine-tuning types. As shown in Table 1, fine-tuning significantly boosted LLM performance. Notably, the Qwen2-0.5B-Instruct model, after multi-task fine-tuning, outperformed the larger Qwen2-7B-Instruct model, demonstrating that smaller LLMs could excel with proper fine-tuning. Multi-task fine-tuning surpassed single-task and mixed-task methods, achieving superior results on each dataset.

- We assessed the effect of interactive output enhancement on accuracy and quality. Table 3 showed that $N_{rounds} = 3$ rounds of dialogue with the auxiliary model notably improved outcomes, especially in text generation tasks like SQuAD, compared to multi-task fine-tuning alone.
- We compared baselines for selection scores with fixed model predictions. Table 2 indicated that our Stitch method significantly outperformed baseline approaches in AUACC and AUROC, with improved results following multi-task fine-tuning and model integration.

Effect of N_{rounds} . We studied how dialogue turns impacted model output quality. Figure 4 and Table 5 showed that more dialogue rounds did not always enhance model performance, with 6 rounds on CommonsenseQA yielding less improvement than 3 rounds.

5.3 Complexity Analysis

We analyzed the complexity of various methods, focusing on the number of forward passes required by the LLM. Since the LLM produced the output sequence in an auto-regressive way, the number of forward passes was proportional to the length of the generated output sequence. Let’s assume the maximum length of the output sequence was denoted as l_{max} . For each input sequence, one forward pass was needed to encode the input, and up to l_{max} forward passes were required to generate the output sequence. Consequently, the upper bound for the number of forward passes was $1 + l_{max}$, giving the complexity as $O(l_{max})$.

For the perplexity method, the complexity remained $O(l_{max})$, as only an additional forward pass was required to compute the perplexity score. As for the semantic entropy method, we excluded the computational cost of the bidirectional entailment clustering algorithm because it was relatively small compared to the cost of generating output from the LLM, as noted in Kuhn et al.. Therefore, the com-

Method	Complexity
Perplexity	$O(l_{max})$
Semantic Entropy	$O(m \cdot l_{max})$
Self-eval	$O(l_{max})$
ASPIRE	$O(l_{max})$
Stitch(ours)	$O(l_{max})$

Table 4: Overview of complexities for different method.

putational complexity for semantic entropy was $O(m \cdot l_{max})$.

For the self-eval method, the complexity was also $O(l_{max})$ since only one extra forward pass was needed to compute the self-eval score. Similarly, ASPIRE had a computational complexity of $O(l_{max})$ because just one more forward pass was required to obtain the learned self-eval score. For our proposed approach, Stitch, the computational complexity was $O(l_{max})$, as we only needed to fine-tune the RoBERTa model and perform a single forward pass to obtain the reliability assessment. Table 4 provided a summary of the complexities for these different methods.

6 Conclusion

In this paper, we introduce a new approach designed to enhance selective prediction in LLMs. This approach leverages multi-task fine-tuning, model integration, and reliability assessment to achieve its goals. Experiments demonstrate that our approach outperforms existing methods. Future work will focus on exploring additional parameter-efficient techniques and improved evaluation metrics to further refine our framework and extend its application to a broader range of LLMs.

Limitations

Our experiments demonstrate that multi-task fine-tuning works best when tasks are within the same domain or closely related, enabling the model to share knowledge and improve performance. Conversely, when tasks are less correlated, the benefits decrease.

Although larger LLMs generally perform better, computational limits prevented us from fine-tuning the largest models. Our framework, however, is adaptable to models of any size, and future work will explore its application to more powerful models.

References

- Abhinav Pandey Abhishek Kadian Abhimanyu Dubey, Abhinav Jauhri et al. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Jiefeng Chen, Jinsung Yoon, Sayna Ebrahimi, Serkan Arik, Tomas Pfister, and Somesh Jha. 2023. Adaptation with self-evaluation to improve selective prediction in llms. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5190–5213.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, et al. 2023. [Mistral 7b](#). *Preprint*, arXiv:2310.06825.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.
- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*.
- Chin-Yew Lin and Franz Josef Och. 2004. Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics. In *Proceedings of the 42nd annual meeting of the association for computational linguistics (ACL-04)*, pages 605–612.
- Bingchang Liu, Chaoyu Chen, Zi Gong, Cong Liao, Huan Wang, Zhichao Lei, Ming Liang, Dajun Chen, Min Shen, Hailian Zhou, et al. 2024. Mftcoder: Boosting code llms with multitask fine-tuning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5430–5441.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. [SQuAD: 100,000+ questions for machine comprehension of text](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.
- Jie Ren, Jiaming Luo, Yao Zhao, Kundan Krishna, Mohammad Saleh, Balaji Lakshminarayanan, and Peter J Liu. 2022. Out-of-distribution detection and selective generation for conditional language models. In *The Eleventh International Conference on Learning Representations*.
- Chenglei Si, Zhe Gan, Zhengyuan Yang, Shuohang Wang, Jianfeng Wang, Jordan Boyd-Graber, and Lijuan Wang. 2023. [Prompting gpt-3 to be reliable](#). In *International Conference on Learning Representations (ICLR)*.
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. [CommonsenseQA: A question answering challenge targeting commonsense knowledge](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.
- Bowen Tan, Yun Zhu, Lijuan Liu, Eric Xing, Zhiting Hu, and Jindong Chen. 2024. Cappy: Outperforming and boosting large multi-task llms with a small scorer. *Advances in Neural Information Processing Systems*, 36.
- Neeraj Varshney, Swaroop Mishra, and Chitta Baral. 2022. [Investigating selective prediction approaches across several tasks in IID, OOD, and adversarial settings](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 1995–2002, Dublin, Ireland. Association for Computational Linguistics.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, et al. 2024. [Qwen2 technical report](#). *Preprint*, arXiv:2407.10671.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. [Llamafactory: Unified efficient fine-tuning of 100+ language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.

A Hardware, Software, and Training Details

We conducted all experiments using the HuggingFace library on a setup with 8 NVIDIA A100 GPUs (80GB each) running Ubuntu. We utilized the HuggingFace Transformers library to work with Qwen2, Mistral, and Llama3 models. Additionally, we employed the Llama-Factory(Zheng et al., 2024) framework for single and mixed fine-tuning. For generating answer sets, we used the default decoding strategy.

For single-task and mixed fine-tuning, we used Lora-based fine-tuning, training for 8 epochs on the CommonsenseQA and Ai2_arc_easy datasets, and 6 epochs on the SQuAD dataset. The AdamW optimizer with a cosine annealing learning rate schedule was employed. Both the training and validation batches had a size of 32, with a learning rate of $1.0e-4$ and gradient accumulation set to 1. The best model was selected based on the validation loss. For multi task fine-tuning, the padding mode was set to "pack," weighted loss mode was configured as case3, early stopping stall was set to 5, lora_rank was set to 96, lora_alpha to 192, and lora_dropout to 0.05. All datasets were trained for 8 epochs, with other hyperparameters kept consistent with those used for single-task and mixed fine-tuning.

B Datasets

We use three question answering datasets: CommonsenseQA (Talmor et al., 2019), Ai2_arc_easy (Clark et al., 2018) and SQuAD (Rajpurkar et al., 2016).

B.1 CommonsenseQA

CommonsenseQA is a new multiple-choice question answering dataset that requires different types of commonsense knowledge to predict the correct answers. It contains 12,102 questions with one correct answer and four distractor answers. We use the following template to construct question queries:

Please answer the Question by choosing the most appropriate answer from the 5 options below.

Question: [Provided question]

Choices:

A. [Option A provided]

B. [Option B provided]

C. [Option C provided]

D. [Option D provided]

E. [Option E provided]

B.2 Ai2_arc_easy

A new dataset of 7,787 genuine grade-school level, multiple-choice science questions, assembled to encourage research in advanced question-answering. The dataset is partitioned into a Challenge Set and an Easy Set, where the former contains only questions answered incorrectly by both a retrieval-based algorithm and a word co-occurrence algorithm. In this paper, we only used an Easy set. We use the following template to construct question queries:

Please answer the Question by choosing the most appropriate answer from the 4 options below.

Question: [Provided question]

Choices:

A. [Option A provided]

B. [Option B provided]

C. [Option C provided]

D. [Option D provided]

B.3 SQuAD

Stanford Question Answering Dataset (SQuAD) is a reading comprehension dataset, consisting of questions posed by crowd-workers on a set of Wikipedia articles, where the answer to every question is a segment of text, or span, from the corresponding reading passage. We use the SQuAD 1.1 version, containing 100,000+ question-answer pairs on 500+ articles. The training set contains 86,821 question queries while the test set contains 5,928 question queries. We use the following template to construct question queries:

Please help me answer the Question based on the Context content. The answers given can only be fragments of the context and you only need to output the answer. Do not output other content including but not limited to explaining the reasons, etc.

Context: [Provided Context]

Question: [Provided question]

In addition, during the data enhancement process, we generated more than 50,000 additional data by calling the LLMs API. The specific prompt is:

I need to perform data augmentation. Can you help me generate some new answers? I will provide a context, a question, and the correct answer to the question.

You need to generate 5 new answers based on this information. At least one answer must be completely wrong, and the rest should be incompletely correct or synonyms of the correct answer. Because I want to save your output, your output format can only be the format in the example and cannot output other irrelevant content. Here is an example:

Context: Super Bowl 50 was an American football game to determine the champion of the National Football League (NFL) for the 2015 season. The American Football Conference (AFC) champion Denver Broncos defeated the National Football Conference (NFC) champion Carolina Panthers 24-10 to earn their third Super Bowl title. The game was played on February 7, 2016, at Levi's Stadium in the San Francisco Bay Area at Santa Clara, California. As this was the 50th Super Bowl, the league emphasized the "golden anniversary" with various gold-themed initiatives, as well as temporarily suspending the tradition of naming each Super Bowl game with Roman numerals (under which the game would have been known as "Super Bowl L"), so that the logo could prominently feature the Arabic numerals 50.

Question: Which NFL team represented the AFC at Super Bowl 50?

Ground Truth: Denver Broncos

Answers you produced: {"answer1": "Obama", "answer2": "Broncos", "answer3": "New England Patriots", "answer4": "Carolina Panthers", "answer5": "Taylor Swift"}

OK, please output the answer that meets the requirements according to the following information

C Baselines

For selective prediction, we need to get a predicted output sequence $\hat{\mathbf{y}}^*$ and a selection score for each input sequence $\mathbf{x}$ given a model f . We consider the following baselines to compute the selection score:

Perplexity. Perplexity is defined as the exponentiated average negative log-likelihood of a sequence. The perplexity of the generated output sequence $\hat{\mathbf{y}}^*$ is computed as:

$$\text{perp}(\hat{\mathbf{y}}^*|\mathbf{x}; f) = f_{\text{norm}}(\hat{\mathbf{y}}^*|\mathbf{x})^{-1} \quad (13)$$

Semantic Entropy. Semantic entropy is an entropy-based uncertainty measure that uses a bidirectional entailment algorithm for marginalizing over semantically-equivalent samples. We follow the settings in Kuhn et al.. Specifically, we use multinomial sampling with a temperature of 0.5 to obtain an answer list of size 4 for each input sequence for uncertainty estimation.

Self-eval. Self-eval is a simple baseline that obtains a selection score from the LLM by asking whether the proposed answer $\hat{\mathbf{y}}^*$ is correct or not. Suppose z_s is a series of tokens representing the self-evaluation trigger string "The answer is ". Suppose z_c and z_w are the tokens that represent the words "correct" and "wrong" respectively. Recall that the logits of the model f on v given $\mathbf{x}$ is $\bar{f}(v|\mathbf{x})$. Then, the self-eval score is computed as:

$$P(z_c|\mathbf{x}, \hat{\mathbf{y}}^*) = \frac{\exp(\bar{f}(z_c|\mathbf{x}, \hat{\mathbf{y}}^*, \mathbf{z}_s))}{\sum_{z \in \{z_c, z_w\}} \exp(\bar{f}(z|\mathbf{x}, \hat{\mathbf{y}}^*, \mathbf{z}_s))}, \quad (14)$$

ASPIRE. ASPIRE(Chen et al., 2023) has two stages of training: the first stage is to train the soft prompt θ_p using the training objective and the second stage is to train the soft prompt θ_s using the training objective:

$$\min_{\theta_p} \mathbb{E}_{(x,y) \sim D^{\text{tr}}} \mathcal{L}(x, y; \theta, \theta_p) \quad (15)$$

$$\min_{\theta_p} \mathbb{E}_{(x,y) \sim D^{\text{tr}}} \mathcal{L}_c + \mathcal{L}_w \quad (16)$$

$$\mathcal{L}_c = \mathbb{E}_{\hat{y} \sim S_c(x,y)} - \log f(z_c|x, \hat{y}; \theta_p, \theta_s) \quad (17)$$

$$\mathcal{L}_w = \mathbb{E}_{\hat{y} \sim S_w(x,y)} - \log f(z_w|x, \hat{y}; \theta_p, \theta_s) \quad (18)$$

We conduct multi-round dialogues by calling the APIs of two llm for interactive output enhancement. We use the following template to construct multi-round dialogues:

Background

Suppose you are [name], you and [opponent_name] will act as encyclopedias with rich knowledge, now I will give the question and the answer, now you need to have a conversation with [opponent_name]

Model	N_{rounds}	Commonsense QA	Ai2_arc_easy	SQuAD	
		Acc $\uparrow$	Acc $\uparrow$	EM $\uparrow$	F1 $\uparrow$
Qwen2-0.5B Instruct	0	0.6192	0.6604	0.6658	0.8095
	3	0.6593	0.8619	0.9259	0.9502
	4	0.6609	0.8632	0.9263	0.9601
	5	0.6618	0.8700	0.9265	0.9274
	6	0.6626	0.8687	0.9265	0.9274
Qwen2-7B Instruct	0	0.8469	0.9491	0.7277	0.8659
	3	0.8608	0.9689	0.9644	0.9779
	4	0.8632	0.9676	0.9632	0.9776
	5	0.8632	0.9642	0.9580	0.9749
	6	0.8616	0.9642	0.9573	0.9743
Mistral-7B Instruct-v0.2	0	0.8280	0.9137	0.7305	0.8681
	3	0.8526	0.9537	0.9557	0.9732
	4	0.8509	0.9541	0.9560	0.9738
	5	0.8575	0.9537	0.9559	0.9740
	6	0.8583	0.9579	0.9574	0.9742
Llama3-8B Instruct	0	0.8215	0.9356	0.7293	0.8674
	3	0.8542	0.9651	0.9570	0.9743
	4	0.8534	0.9638	0.9556	0.9733
	5	0.8526	0.9621	0.9545	0.9730
	6	0.8526	0.9630	0.9552	0.9735

Table 5: Performance comparison across different models and datasets over multiple rounds of interaction. **Bold** numbers are superior results.

based on your rich knowledge and history
To improve the answer, you need to give
a reasonable answer after a friendly
exchange.

Related Information:
[question]
[answer]

Historical Conversations
[context]

rule:
1. During the communication process,
corresponding opinions and reasons should
be given for the other party's reference.
2. Give a reasonable answer after final
negotiation. If you think the user's
answer is already perfect, there is no
need to improve it. The questions are
multiple choice, meaning you can only
choose one answer.
3. Because I want to extract your output
content, for the sake of uniformity,
please ensure that the output format is
{"communication content": "xx", "complete
answer": "xx"}. In addition, the value
of the "complete answer" can only be one
of the options A, B, C, D, E.